GUI

- GUI stands for Graphical User Interface.
- It gives programmers an easy to use visual experience.
- It is mainly made of graphical components like buttons, labels, windows, etc. through which the user can interact with the applications.
- **Swing and AWT** are widely used API in Java.
- Before Swing, Abstract Window Toolkit (AWT) was used to build GUI in Java

Context	AWT	Swing
API Package	The AWT Component classes are provided by the java.awt package.	The Swing component classes are provided by the javax.swing package.
Operating System	The Components used in AWT are mainly dependent on the operating system.	The Components used in Swing are not dependent on the operating system. It is completely scripted in Java.
Weightiness	The AWT is heavyweight since it uses the resources of the operating system.	The Swing is mostly lightweight since it doesn't need any Operating system object for processing. The Swing Components are built on the top of AWT.
Appearance	The Appearance of AWT Components is mainly not configurable. It generally depends on the operating system's look and feels.	The Swing Components are configurable and mainly support pluggable look and feel.
Number of Components	The Java AWT provides a smaller number of components in comparison to Swing.	Java Swing provides a greater number of components than AWT, such as list, scroll panes, tables, color choosers, etc.
Memory	Java AWT needs a higher amount of memory for the execution.	Java Swing needs less memory space as compared to Java AWT.
Speed	Java AWT is slower than swing in terms of performance.	Java Swing is faster than the AWT.

Swing is having two key items

- Components
- Containers

Components

- A component is an independent visual control
- Components are the basic visual elements that users interact with in a GUI application.
- They can represent buttons, labels, text fields, checkboxes, radio buttons, list boxes, sliders, and more. Each component has a specific function and behavior associated with it. For example:
    - JButton: Represents a clickable button.
    - JLabel: Displays a non-interactive text or an image.
    - JTextField: Provides an input field for the user to enter text.
    - JCheckBox: Represents a checkbox that can be checked or unchecked.
- All these components are lightweight components.
- Components can handle events, such as user clicks or keyboard input, and respond accordingly.

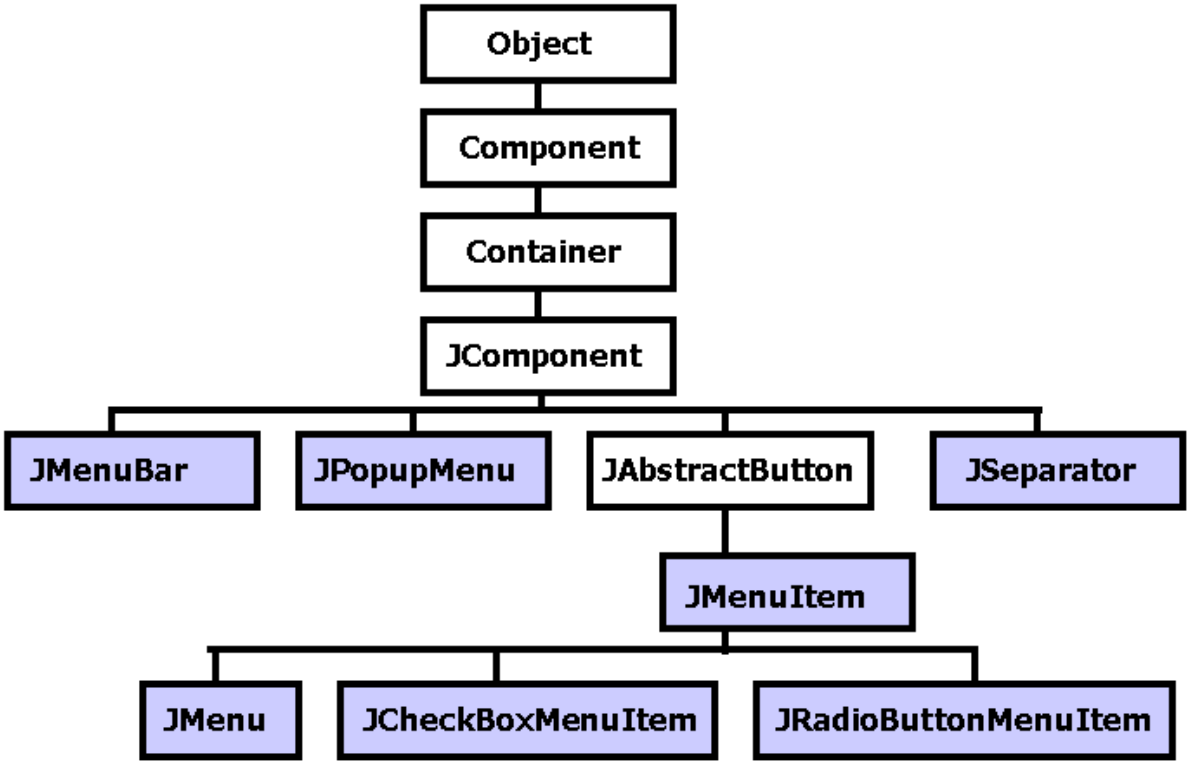
Containers

- A container holds a group of components. It provides a space where a component can be managed and displayed.
- Containers are objects that provide structure and organization for components in a GUI application.
- They are responsible for layout management, positioning components, and determining how they interact with each other.
- Containers can be thought of as invisible boxes that hold components. There are different types of containers with varying layout behaviors:

- JFrame: Represents the main window of a GUI application.
- JPanel: used to organize and group components within a JFrame.

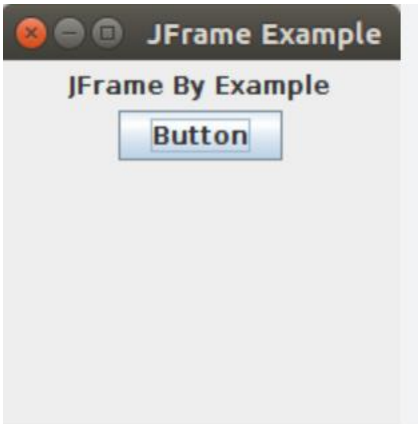
- Containers are of two types:
  1. **Top-level containers**
     - Top-level containers are the highest-level containers in a GUI application.
     - They represent the main windows or frames that serve as the top-level parent for other components.
     - These containers are usually the main entry point of the application and are responsible for providing the application's primary window or window-like interface.
     - JFrame, JWindow, JDialog etc
     - These containers do not inherit JComponent.
     - But they inherit the AWT classes Component and Container.
  2. **Lightweight Containers**
     - Lightweight containers are used to organize and hold components within a top-level container or other lightweight containers.
     - It inherits JComponent class.
     - It can be used to organize related components together.
     - Example: JPanel

Swing Hierarchy



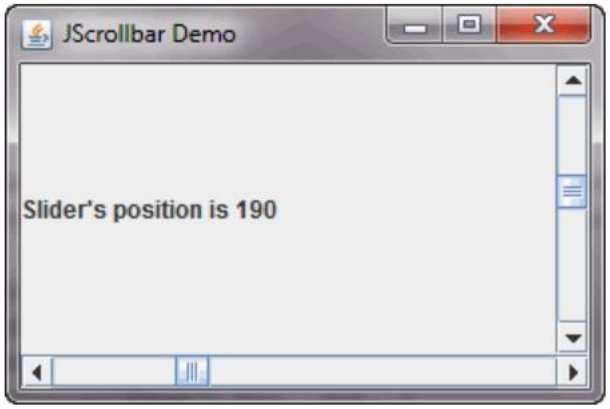
1. Jframe

- JFrame class is a type of container which inherits the java.awt.Frame class.
- JFrame works like the main window where components like labels, buttons, textfields are added to create a GUI.



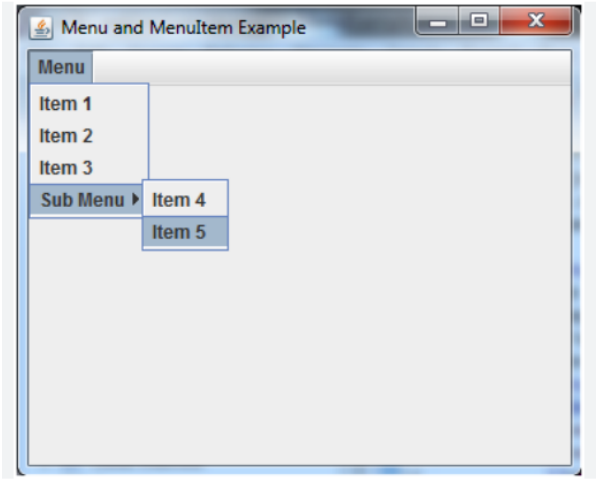
2. JScrollBar

- The object of JScrollbar class is used to add horizontal and vertical scrollbar.
- It inherits JComponent class.



3. JMenuBar

- The JMenuBar class is used to display menubar on the window or frame. It may have several menus.
- The object of JMenu class is a pull down menu component which is displayed from the menu bar. It inherits the JMenuItem class.



4.JPanel

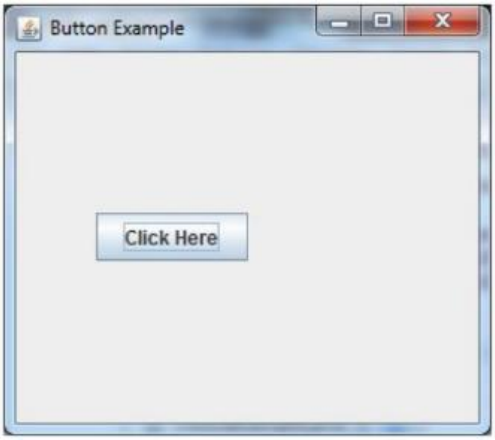
The JPanel is a simplest container class. It provides space in which an application can attach any other component

5.JButton

JButton-is a fundamental Swing component that renders a button on screen and responds to user's clicking event for performing a specific task.

Commonly used Constructors:

- JButton()- It creates a button with no text and icon.
- JButton(String s)- It creates a button with the specified text.
- JButton(Icon i)- It creates a button with the specified icon object



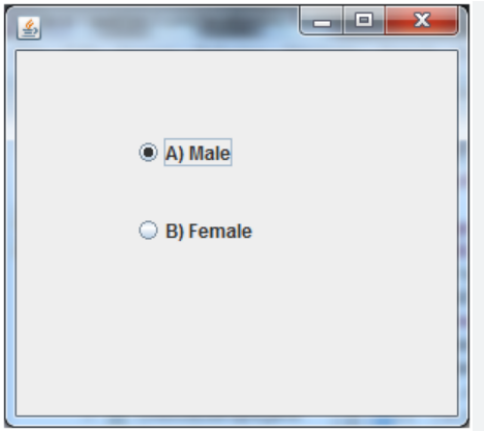
6. JLabel

Jlabel is a class from the **swing** package is able to display a text or a picture or both.

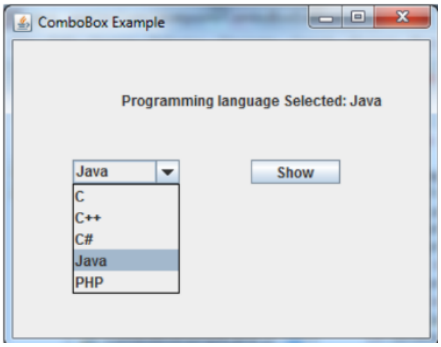
7. JTextField is a fundamental Swing's component that allows users editing a single line of text

8. JPasswordField - allows the editing of a single line of text where the view indicates something was typed, but does not show the original characters

9. JRadioButton class is used to create a radio button. It is used to choose one option from multiple options. It is widely used in exam systems or quiz.



10. JComboBox shows a popup menu that shows a list and the user can select a option from that specified list



Events

- Change in the state of an object is known as Event.
- Events are generated as a result of user interaction with the graphical user interface components.
- Clicking on a button, moving the mouse, entering a character through keyboard, selecting an item from the list, and scrolling the page are the activities that causes an event to occur.

- event types are defined in package javax.swing.event.
- Two types: Foreground Events, Background Events.

1. **Foreground Events**
   - These events require direct interaction of the user.
   - They are generated as consequences of a person interacting with the graphical components in the Graphical User Interface.
   - Eg. clicking on a button, moving the mouse.

2. Background Events

- Events that occur independently of user interactions.
- These events are often related to background processes, timers, or other system events.
- Eg. Operating system interrupts, hardware or software failure.

Event Handling

- **Event Handling**: allows your program to respond to various events, such as button clicks, key presses, mouse actions, etc. When an event occurs, the associated event handler is triggered to perform a specific task or action.

Event Handler:

- An event handler is a block of code that is executed in response to a specific event. It contains the logic that should be performed when the event occurs.

- Key terms: Source, Listener.

Source

- The source is an object on which the event occurs.
- Source is responsible for providing information of the occurred event to it's handler.
- For example, if a button is clicked, the button is the source of the click event.

Listener

- It is also known as event handler.
- The listener is responsible for generating a response to an event.

- **Listeners** are used for handling the events generated from the source. Each of these listeners represents interfaces that are responsible for handling events.

- To perform **Event Handling**, we need to register the source with the listener. When an event occurs, the GUI component with which the user interacted notifies its registered listener by calling each listener's appropriate event handling method.

Event Class	Event Listener (Interface)
ActionEvent	ActionListener
ItemEvent	ItemListener
MouseEvent	MouseListener, MouseMotionListener
WindowEvent	WindowListener
KeyEvent	KeyListener

Example

- A command button generates an ActionEvent when the user clicks the button with the mouse. Here for the event to be generated on button click, we need to register the button with the Action Listener.
- The only method in ActionListener interface is actionPerformed which need to be implemented. So we write the event that should happen in the actionPerformed method.